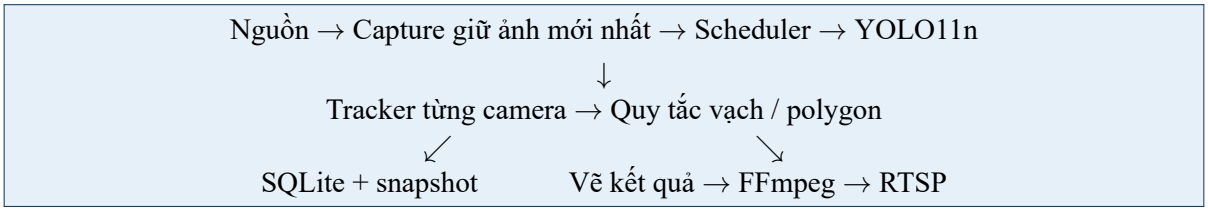
BÁO CÁO KỸ THUẬT
HỆ THỐNG XỬ LÝ VIDEO ĐA CAMERA

Người thực hiện: Trần Danh Thiện
github.com/thien2603/Detect-and-Tracking
Cập nhật 26/10/2026; benchmark đo ngày 23/10/2026.

Kết quả chính: chạy đồng thời 4 nguồn, nhận diện người/xe, tracking, đếm hai chiều và cảnh báo vùng. Microbatch đạt trung bình **11,07 FPS ảnh mới/camera**; độ trễ lớn nhất **1129 ms**, chỉ 3/4 camera đạt dưới 1 giây. Chưa đạt toàn bộ tiêu chí hiệu năng.

1 Kiến trúc và luồng hoạt động

Bốn nguồn gồm ba góc Bellevue và một góc Crosswalk có người đi bộ. Khi benchmark, file video được phát lại qua RTSP; mỗi camera có tracker, bộ đếm và đường RTSP đầu ra riêng.



Capture độc lập giúp một nguồn lỗi không kéo các nguồn khác dừng theo. Scheduler giữ tối đa một yêu cầu chờ/camera; ảnh cũ được thay bằng ảnh mới. Ghi bằng chứng và phát RTSP tách khỏi suy luận. FastAPI cung cấp trạng thái, FPS, bật/tắt camera và truy vấn sự kiện.

Nhóm mã nguồn	Chức năng
`app/camera/capture.py`	Đọc file/RTSP/webcam, latest-frame, reconnect.
`app/inference/`	YOLO, warmup và điều phối batch.
`app/inference/tracker.py`; `motion.py`	IoU, Kalman và ID riêng theo camera.
`app/pipeline.py`; `app/events/`	Quy tắc đếm/vùng; SQLite, snapshot.
`app/streaming/`; `app/manager.py`	Overlay, FFmpeg và vòng đời camera.
`app/main.py`; `configs/`	API và cấu hình YAML.

2 Thuật toán và quy tắc sự kiện

Detection/tracking: YOLO11n tìm bbox và lớp đối tượng. Tracker ghép bbox cùng lớp bằng IoU; bản tối ưu dùng Kalman dự đoán vị trí trước khi ghép, hỗ trợ chuyển động nhanh. Che khuất hoặc thiếu detection lâu vẫn có thể gây đổi ID.

Đếm qua vạch: lấy tâm cạnh dưới bbox; tăng bộ đếm theo chiều khi điểm đổi phía và cắt đoạn vạch hữu hạn, có deadband/cooldown.

Vào vùng: kiểm tra điểm chân hộp trong polygon; ghi sự kiện khi đi vào, không ghi lại mỗi frame nếu đứng trong vùng. Lý do `first_seen_inside` được lưu riêng khi lần đầu thấy đã ở trong vùng. Retry cùng event ID không chèn trùng vào SQLite.

Giới hạn: dự đoán chỉ để vẽ; sự kiện dùng bbox YOLO đo thật. Hộp quá tuổi 700 ms bị ẩn. Chưa có ground truth để công bố accuracy; đổi track ID có thể gây đếm lại cùng vật thể.

3 Cấu hình và yêu cầu chức năng

Cấu hình chính: `configs/submission-optimized.yaml`. Vạch/polygon được căn thủ công theo từng góc camera cố định, lưu tọa độ pixel trong YAML. Vùng xanh khoảng $3/4$ và đỏ khoảng $1/4$ ROI là vùng minh họa, chưa quy đổi thành diện tích thực.

Tham số	Giá trị trong ma trận 1/4/8 camera
Model / thiết bị	YOLO11n, CUDA RTX 3050 Laptop, FP16.
Ảnh / confidence	`imgsz=480`, ảnh vuông; confidence 0,25.
Lớp	person, car, motorcycle, bus, truck.
Điều phối	Serial: batch 1; microbatch: tối đa 4, chờ 15 ms.
Render / detect	Bất đồng bộ; `detect_interval=2`; Kalman/IoU.
RTSP	H.264 NVENC, mục tiêu 14 FPS, tối đa 960×540, giữ tỷ lệ.
Vòng đời	Hàng đợi hữu hạn, bỏ ảnh quá cũ, tự khởi động lại FFmpeg.

Mã	Yêu cầu đề bài	Kết quả đối chiếu
F01	File/RTSP/webcam; reconnect; cô lập lỗi	File/RTSP đã chạy; reconnect có test tích hợp. Webcam có code, chưa thử thiết bị thật.
F02	Luôn xử lý frame mới nhất	Có latest-frame slot, queue hữu hạn và test thay yêu cầu cũ.
F03	YOLO v8+, người/xe; class/conf/imgsz từ config	YOLO11n; có người/xe thật trong nguồn; tham số YAML.
F04	Tracking	ID theo camera; test cùng lớp, mất quan sát ngắn và chuyển động nhanh.
F05	Đếm qua vạch hai chiều	Unit test vạch hữu hạn, deadband/cooldown; demo có line crossing.
F06	Phát hiện vào polygon	Test biên, entry/stay/exit/reentry; demo có intrusion.
F07	Một event/lần vi phạm, SQLite và ảnh	Có UTC, camera, track, loại và snapshot; retry không chèn trùng.
F08	Overlay, RTSP riêng, VLC, FFmpeg restart	RTSP decode thật, overlay và restart có kiểm thử; chưa nghiệm thu VLC GUI.
F09	YAML; fail-fast; thêm camera bằng config	Schema/preflight trước khi chạy; test cấu hình sai và thêm nguồn.
F10	FastAPI trạng thái/FPS, bật/tắt, lọc event	Có endpoint và test API, lọc camera/thời gian, snapshot.

API chính: `GET /api/v1/cameras`; `PUT /api/v1/cameras/{id}/enabled`; `GET /api/v1/events`. Tài liệu tương tác tại `/docs` khi service đang chạy.

Điểm cần phân biệt: có đầy đủ các nhóm chức năng trong mã nguồn, nhưng F01 còn thiếu nghiệm thu webcam vật lý, F08 còn thiếu mở bằng VLC. Độ chính xác đếm/tracking chưa được đo bằng nhãn chuẩn; đề không đặt ngưỡng accuracy cụ thể.

4 Benchmark và lựa chọn kiến trúc

Máy đo: Lenovo Legion 5 15ARH7; Ryzen 5 6600H (6 nhân/12 luồng); RAM lắp 8 GB, dùng được khoảng 7,19 GiB; RTX 3050 Laptop 4 GB, driver 546.92. Windows build 26200, Python 3.10.11, PyTorch CUDA, MediaMTX 1.9.3.

Mỗi phương án đo 60 giây sau 8 giây warmup. Tám camera là tám luồng logic lấy từ bốn video. **Detect FPS** đếm ảnh YOLO hoàn tất; **unique FPS** đếm timestamp ảnh nguồn khác nhau giải mã từ RTSP đầu ra. Render bất đồng bộ giúp ảnh xem mới hơn nhịp detect.

Độ trễ đo từ replay tới client giải mã RTSP trên cùng máy; chưa gồm hiển thị VLC và phơi sáng camera vật lý. Cột trễ đạt dùng max < 1000 ms và marker coverage $\geq 95\%$. FPS là trung bình camera; RAM là peak RSS cộng các tiến trình, có thể tính trùng trang dùng chung.

Cam	Cách chạy	Detect	Unique	Max ms	Trễ đạt	RAM MiB
1	Serial	6,48	12,55	639	1/1	1555
1	Microbatch	6,52	12,38	723	1/1	2036
4	Serial	1,14	10,52	1091	1/4	1964
4	Microbatch	3,05	11,07	1129	3/4	2583
8	Serial	0,17	0,91	10191	0/8	1426
8	Microbatch	0,37	0,98	7277	0/8	1582

Nhận xét: 1 camera đạt FPS và độ trễ. 4 camera đạt unique FPS ≥ 10 cho từng camera, nhưng chưa đạt độ trễ cho tất cả. 8 camera quá tải; một số luồng gặp lỗi encoder NVENC và restart, nên kết quả phản ánh cả giới hạn xử lý/phát video.

Chọn shared microbatch: ở 4 camera, detect tăng từ 1,14 lên 3,05 FPS/camera; CPU tổng giảm từ 702% xuống 646%; số camera đạt độ trễ tăng từ 1/4 lên 3/4. CPU cộng theo tiến trình có thể vượt 100% trên máy nhiều nhân. Max 1129 ms vẫn chưa đạt ngưỡng đề.

Đối chứng riêng cùng 4 nguồn, FP32 và libx264 cho bảng dưới. Cấu hình này khác bảng trên, chỉ dùng so sánh số model trong cùng phép thử.

Phương án	Detect FPS/cam	CPU %	RSS MiB
Một model chung / batch	5,01	576	1937
Bốn model riêng	4,82	675	1991
Bốn model + CUDA stream riêng	4,94	676	2008

Bốn model trên cùng GPU chưa cải thiện tốc độ; vì vậy giữ model chung. Mỗi cấu hình mới đo một lượt, chưa có kiểm tra dài hạn hoặc khoảng tin cậy.

Bằng chứng: hướng dẫn và số liệu trong repository:

`docs/benchmark-rerun-2026-10-23.md`

`docs/model-modes-2026-10-23.md`

JSON thô giữ local (thư mục data không đưa lên GitHub):

`data/benchmark-rerun-20261023-01/`

`data/benchmark-rerun-20261023-4-8-02/`

5 Kiểm thử và đối chiếu nghiệm thu

Test kiểm tra tracking, vạch/vùng, event, API và vòng đời tài nguyên. Integration dùng FFmpeg/MediaMTX thật, ngắt/nối lại nguồn và dừng FFmpeg để thử restart. Detector giả dùng trong test hạ tầng; benchmark hiệu năng dùng YOLO thật.

Kiểm tra ngày 26/10/2026: 94 test chính và 7 subtest đều pass, không lỗi/bỏ qua; Ruff đã qua. JUnit được lưu trong `docs/verification/pytest-20261026.xml`. Docker đã build/up và kiểm tra thật trên CPU với profile riêng; bảng benchmark GPU ở trang 3 giữ nguyên.

Docker CPU: `configs/docker-cpu.yaml` dùng ảnh 320, render tối đa 640×360, mục tiêu 10 FPS, replay 10 FPS. Giữ nguyên tọa độ vạch/vùng và ngưỡng bỏ ảnh cũ 700 ms. Kiểm tra cuối: 4 RTSP decode được; API có 11 sự kiện mới, snapshot, lọc và bật/tắt camera; ba container thoát mã 0, không OOM. Đây là kiểm tra chức năng, chưa phải phép đo chứng minh đạt FPS/latency.

Tiêu chí	Kết luận	Căn cứ / phần còn thiếu
4 camera, mỗi camera ≥ 10 FPS	Đạt đầu ra	Benchmark 4/4 theo unique FPS; demo 120 giây.
Trễ đầu-cuối < 1 giây	Chưa đạt đủ	Microbatch 3/4; max 1129 ms. Chưa đo tới VLC.
Ghi rõ máy và cách đo	Đạt	Phần cứng, cấu hình, phạm vi đo ở trang 3.
Ctrl+C dừng sạch	Có bằng chứng	Cleanup các lượt hoàn tất: thoát mềm, không còn tiến trình con.
Logging file, FPS/trễ/reconnect	Đạt mã	Logging xoay vòng, số liệu theo camera.
Unit test vạch và polygon	Có kiểm thử	Test biên, hai chiều, cooldown, entry/reentry.
Sử dụng Docker	Đạt chạy thực	Build/up, API, 4 RTSP, event/snapshot, hot-toggle và shutdown đều qua trên CPU.
Đo 1/4/8, ít nhất hai cách	Đạt đo đạc	Sáu tổ hợp serial/microbatch, 60 giây/lượt. Đề không yêu cầu 8 camera đạt 10 FPS.
Webcam và mở RTSP bằng VLC	Còn xác minh	Có code webcam và RTSP decode thật; thiếu thiết bị/VLC.

6 Hồ sơ nộp và việc còn lại

Mã nguồn: đã xác minh và push lên GitHub. Bản `5c6b843` là mốc trước bổ sung Docker; cấu hình CPU và bằng chứng mới đi kèm lần cập nhật này. Repository lưu source/config/test; model và video tải riêng.

Demo: `Demo_TranDanhThien.mp4`, khoảng 120 giây, ghép bốn nguồn đang chạy đồng thời. Có 391 sự kiện trong khoảng ghi hình: 319 intrusion, 56 region entry, 16 line crossing. Số sự kiện không tương đương số đối tượng duy nhất. Đã giải mã toàn bộ clip để kiểm tra.

Việc chưa hoàn tất:

- Giảm độ trễ camera còn lại xuống dưới 1000 ms và đo lại cùng điều kiện.
- Thử webcam thật; mở bốn RTSP đầu ra bằng VLC.
- Upload PDF/MP4 vào thư mục Drive `TranDanhThien`, kiểm tra quyền xem.

Kết luận: chức năng chính, benchmark và Docker đã có bằng chứng. Còn thiếu độ trễ đạt cho cả 4 camera, nghiệm thu webcam/VLC và xác minh nộp Drive. Chi tiết Docker: `docs/docker-validation-2026-10-26.md`.